KOENIG

step forward



Collections

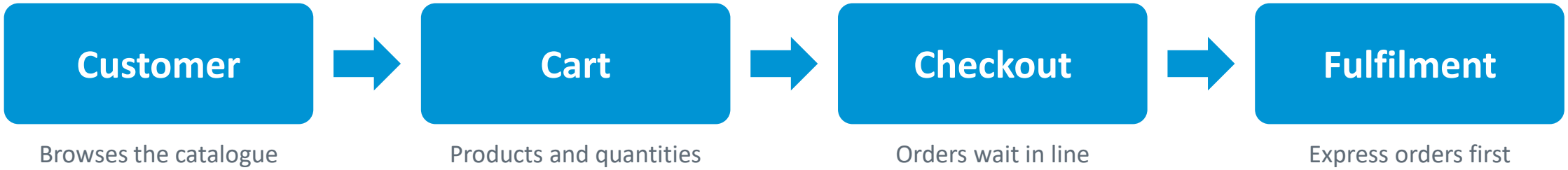
Core Java (Advanced)

What You Will Learn

1. Collection and Map hierarchies
2. Choosing the right implementation
3. Sorting: Comparable and Comparator
4. Iterators and fail-fast behaviour
5. Immutable and unmodifiable collections
6. Concurrent collections overview

Demo Scenario: ShopKart Catalog and Cart

Module 1 gave ShopKart its model: products, customers and money. Now we keep it in the right collections. ShopKart is an online shop. We list products, group them, queue the orders and keep a cart history, using the right collection for every job.

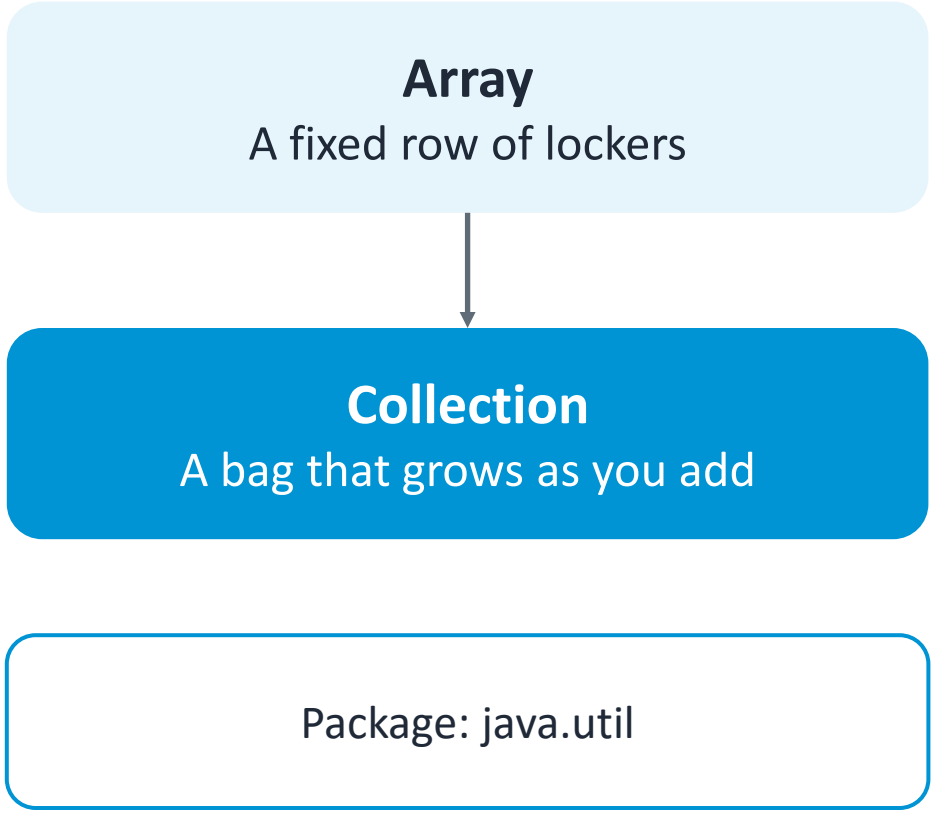


Supporting cast

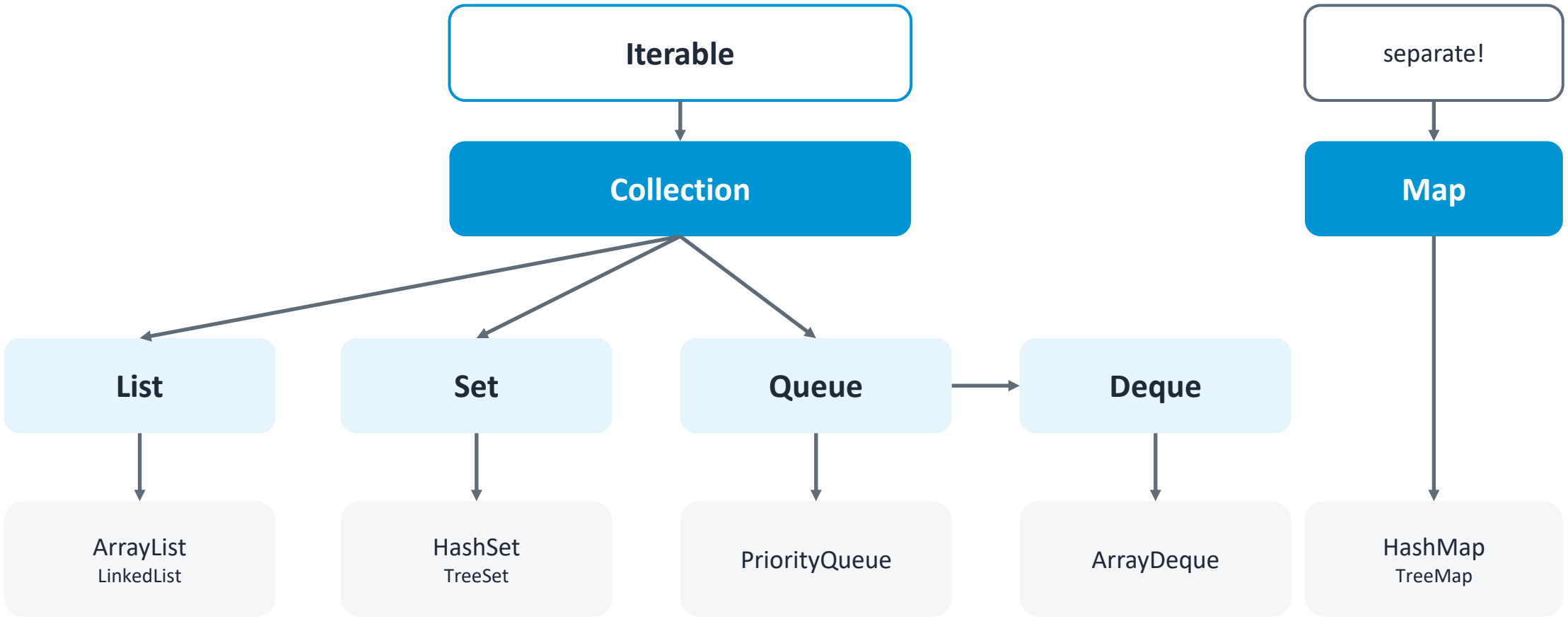
- **Product**
- **Category**
- **Wishlist**
- **Order**

Why a Collections Framework?

- **Arrays:** fixed size, no add or remove, no search helpers.
- **Collections:** grow and shrink automatically.
- **One vocabulary:** add, remove, contains, size, iterate on every type.
- **Tested:** fast, proven algorithms you do not write yourself.



The Collection Hierarchy



Map is NOT a Collection: it stores key-value pairs

Five Interfaces, Five Jobs

1. **List**
   Ordered, allows duplicates, access by index.

2. **Set**
   No duplicates. Think: a guest list.

3. **Queue**
   Waiting line: first in, first out.

4. **Deque**
   Double-ended: add or remove at both ends.

5. **Map**
   Look up a value by its key, like a phone book.

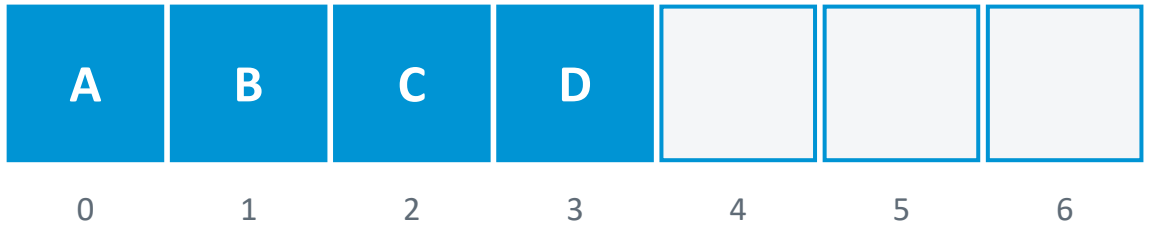
List: Ordered with Duplicates

- **Order kept:** items stay where you put them.
- **Index:** get(0), set(1, x), add(2, x).
- **Duplicates:** allowed.
- **Declare by interface:** so you can swap the implementation.

```
List<String> names = new ArrayList<>();
names.add("Asha");
names.add("Bob");
names.add("Asha");          // duplicate is fine
names.add(1, "Chitra");     // insert at index 1

System.out.println(names);
//  [Asha, Chitra, Bob, Asha]
System.out.println(names.get(2));        // Bob
System.out.println(names.indexOf("Asha")); // 0
names.remove("Bob");
System.out.println(names.size());        // 3
```

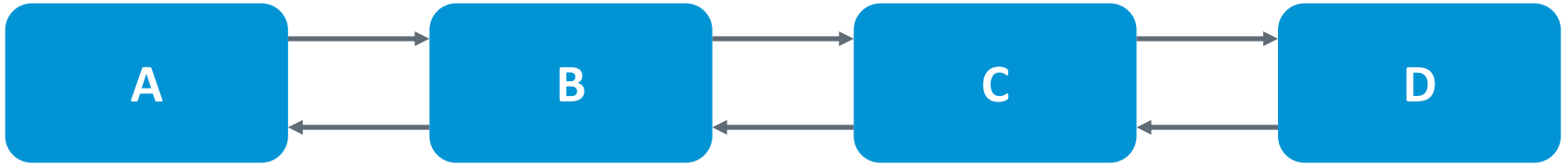
ArrayList: A Growing Array



Full? Make a bigger array
Copy everything over (about 1.5x)

- **get(i):** jump straight to the slot, very fast.
- **add at end:** usually fast.
- **add or remove in the middle:** everything after it shifts.
- **Great for:** reading and looping. The default List.

LinkedList: A Chain of Nodes



- **Each node:** holds a value plus links to next and previous.
- **Insert or remove at the ends:** just re-link, no shifting.
- **get(i):** must walk the chain from an end, slow.
- **Memory:** extra object and two links per item.
- **Also a Deque:** addFirst, removeLast, and so on.

Demo: Lists and Linked Lists

What we build

- **Catalogue:** add, insert at an index, get, indexOf and remove.
- **Recently viewed:** a LinkedList that keeps the newest three products.
- **Declared by interface:** the variable type is List, not ArrayList.

How the concept helps

- **List** keeps order and allows duplicates, with access by index.
- **ArrayList** is the default for browsing and looping.
- **LinkedList** is cheap when adding or removing at the ends.

Set: No Duplicates

HashSet

Fastest. No order guarantee.

LinkedHashSet

Remembers insertion order.

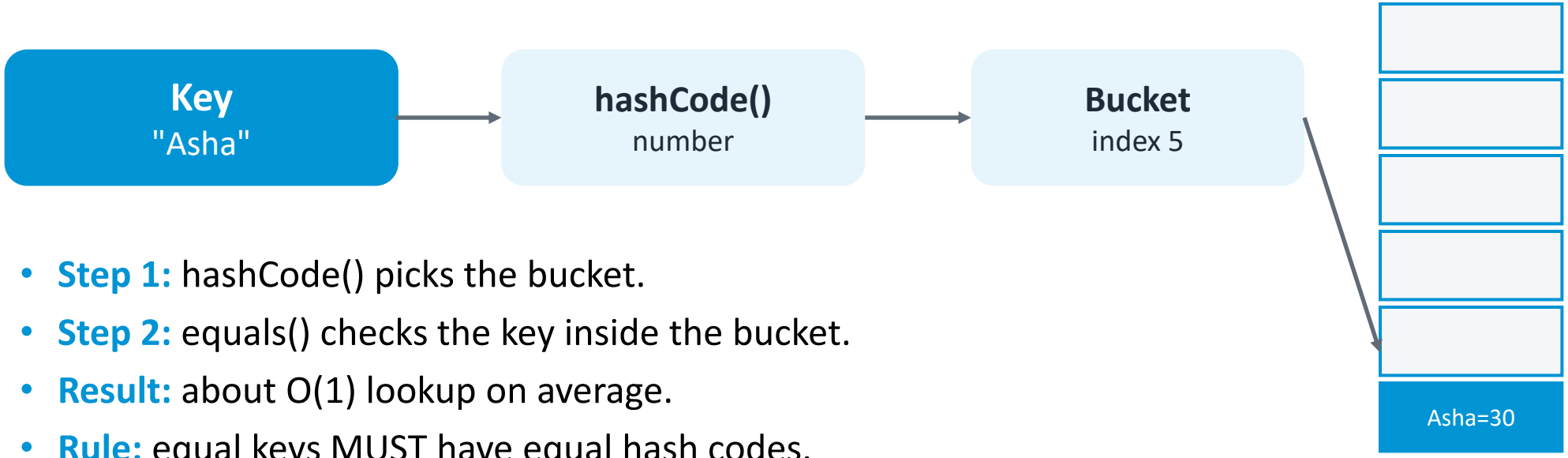
TreeSet

Always sorted.

```
Set<String> tags = new HashSet<>();
System.out.println(tags.add("java"));   // true
System.out.println(tags.add("java"));   // false, already there
tags.add("spring");
System.out.println(tags.contains("java")); // true

Set<String> sorted = new TreeSet<>(tags);
System.out.println(sorted);             // [java, spring]
```

How HashMap Finds Things Fast



- **Step 1:** hashCode() picks the bucket.
- **Step 2:** equals() checks the key inside the bucket.
- **Result:** about O(1) lookup on average.
- **Rule:** equal keys MUST have equal hash codes.
- **Use immutable keys:** String, Integer, records.

Map: Key to Value

- **HashMap:** fast, no order.
- **LinkedHashMap:** insertion order (or access order).
- **TreeMap:** sorted by key.
- **One value per key:** put() replaces the old one.

```
Map<String, Integer> stock = new HashMap<>();
stock.put("pen", 10);
stock.put("book", 4);
stock.put("pen", 12);                // replaces 10

System.out.println(stock.get("pen"));          // 12
System.out.println(stock.getOrDefault("cup", 0));  // 0
stock.merge("book", 1, Integer::sum);          // 5

for (var e : stock.entrySet()) {
    System.out.println(e.getKey() + "=" + e.getValue());
}
Map<String, Integer> sorted = new TreeMap<>(stock);
```

Demo: Sets and Maps

What we build

- **Product tags:** HashSet, LinkedHashSet and TreeSet side by side.
- **Orders by customer:** a Map where Customer is the key.
- **Stock:** merge() restocks, getOrDefault() avoids null, TreeMap shows it sorted.

How the concept helps

- **Set** rejects duplicates, so a tag appears once.
- **Map** finds a customer's orders by key in one step.
- **equals and hashCode** on the key decide whether two customers are the same.

Queue and Deque

- **Queue:** offer() adds, poll() removes the head.
- **PriorityQueue:** head is the smallest by ordering.
- **ArrayDeque:** fast queue AND stack.
- **Prefer ArrayDeque** over the old Stack class.

```
Queue<String> line = new ArrayDeque<>();
line.offer("Asha");
line.offer("Bob");
System.out.println(line.poll());    // Asha (first in)
System.out.println(line.peek());    // Bob (look only)

Deque<String> stack = new ArrayDeque<>();
stack.push("page1");
stack.push("page2");
System.out.println(stack.pop());    // page2 (last in)

Queue<Integer> pq = new PriorityQueue<>();
pq.add(5); pq.add(1); pq.add(3);
System.out.println(pq.poll());      // 1
```

Choosing an Implementation

I need...	Use
Default list, read by index	**ArrayList**
Add/remove at both ends	**ArrayDeque or LinkedList**
Fast lookup by key	**HashMap**
Keep insertion order of a map	**LinkedHashMap**
Sorted by key	**TreeMap**
Unique items, fastest	**HashSet**
Unique items, sorted	**TreeSet**
Always take the most urgent	**PriorityQueue**

When unsure: ArrayList, HashMap, HashSet

Demo: Queues and Deques

What we build

- **Checkout line:** an ArrayDeque used as a first-in, first-out line.
- **Packing:** a PriorityQueue that packs express orders first.
- **Back button:** an ArrayDeque used as a stack of visited pages.

How the concept helps

- **Queue** serves shoppers in the order they arrived.
- **PriorityQueue** serves by urgency, whatever the arrival order.
- **Deque** gives one fast class for both queue and stack jobs.

Comparable: The Natural Order

- **One built-in order** per class.
- **compareTo(other):** negative, zero or positive.
- **Used by** Collections.sort, TreeSet, TreeMap.
- **String, Integer, LocalDate** already have one.

```
public record Student(String name, int marks)
        implements Comparable<Student> {

    @Override
    public int compareTo(Student other) {
        return Integer.compare(this.marks, other.marks);
    }
}

List<Student> list = new ArrayList<>(List.of(
    new Student("Asha", 80), new Student("Bob", 65)));
Collections.sort(list);   // Bob, Asha
```

Comparator: Any Order You Like

- **Separate object** describes an order.
- **Many per class:** by name, by marks, by date.
- **Does not change** the class itself.
- **Pass it to** sort(), TreeSet, PriorityQueue.

```
List<Student> list = ...;

list.sort(Comparator.comparing(Student::name));
list.sort(Comparator.comparingInt(Student::marks));

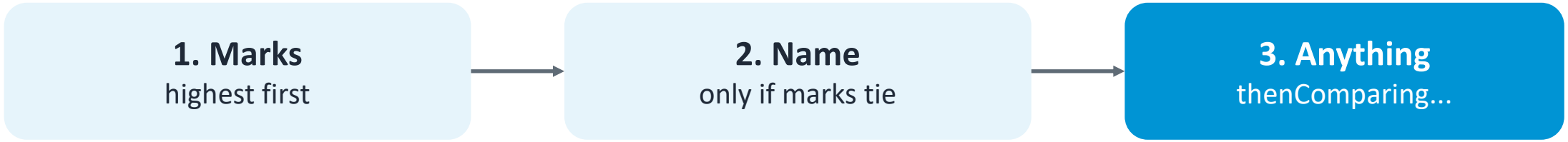
// highest marks first
list.sort(Comparator.comparingInt(Student::marks)
        .reversed());

var byName = new TreeSet<Student>(
        Comparator.comparing(Student::name));
```

Chaining Comparators

```
Comparator<Student> order =
    Comparator.comparingInt(Student::marks).reversed() // 1st
        .thenComparing(Student::name);                 // 2nd

students.sort(order);
```



- thenComparing() runs only when the earlier step says 'equal'.
- Handy extras: nullsFirst(), naturalOrder(), reverseOrder().

Demo: Comparable and Comparator

What we build

- **Natural order:** Product compares by id with Comparable.
- **Other orders:** cheapest first and by name, using Comparator.
- **Shelf order:** category, then price high to low, then name, chained.

How the concept helps

- **Comparable** gives a class one built-in order.
- **Comparator** adds as many orders as we need without changing the class.
- **thenComparing** breaks ties in a clear, readable chain.

Iterator: Walking a Collection

- **hasNext():** is there another item?
- **next():** give me the item, move on.
- **remove():** delete the last item returned.
- **Every Collection** can hand out an Iterator.

```
List<String> names = List.of("Asha", "Bob", "Chitra");

Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    System.out.println(name);
}

// for-each is shorthand for the same thing
for (String name : names) {
    System.out.println(name);
}
```

ListIterator: Both Directions

- **Only for List.**
- **Move both ways:** next() and previous().
- **Change while walking:** set(), add(), remove().
- **Knows the position:** nextIndex().

```
List<String> names =
    new ArrayList<>(List.of("asha", "bob", "chitra"));

ListIterator<String> it = names.listIterator();
while (it.hasNext()) {
    String n = it.next();
    it.set(n.toUpperCase());     // replace in place
}
System.out.println(names);       // [ASHA, BOB, CHITRA]

while (it.hasPrevious()) {       // walk backwards
    System.out.println(it.previous());
}
```

Fail-Fast and the CME

```java
var names = new ArrayList<>(
    List.of("Asha", "Bob", "Bala"));

for (String name : names) {
    if (name.startsWith("B")) {
        names.remove(name);   // BOOM
    }
}
```

ConcurrentModificationException

```java
Iterator<String> it = names.iterator();
while (it.hasNext()) {
    if (it.next().startsWith("B")) {
        it.remove();      // safe
    }
}

names.removeIf(n -> n.startsWith("B"));
```

Works: the iterator knows about the change

- Fail-fast: the collection counts its changes and the iterator throws if the count moved behind its back.

Demo: Iterators and Fail-Fast

What we build

- **Walking:** Iterator, with for-each.
- **ListIterator:** apply a 10% sale to prices in place, then walk back.
- **The bug:** removing out-of-stock items inside for-each throws ConcurrentModificationException, then we fix it.

How the concept helps

- **Iterator** is the one way to walk any collection.
- **Iterator.remove and removeIf** delete safely while looping.
- **Fail-fast** turns a hidden bug into an error you can see.

Unmodifiable vs Immutable

Unmodifiable VIEW

Collections.unmodifiableList(list)

You cannot change it, but the owner still can and the view shows those changes.

Immutable collection

List.of(...), Set.of(...), Map.of(...), List.copyOf(list)

Nobody can change it. No nulls allowed.

```
List<String> view = Collections.unmodifiableList(books);
view.add("x");              // UnsupportedOperationException

List<String> fixed = List.of("A", "B");
fixed.set(0, "Z");          // UnsupportedOperationException
List<String> copy = List.copyOf(books);  // independent snapshot
```

Thread-Safe Collections

- **Problem:** ArrayList and HashMap break if two threads change them together.
- **Old way:** Collections.synchronizedList(...), one lock for everything.
- **Better:** java.util.concurrent classes built for sharing.

ConcurrentHashMap

Thread-safe map, many readers and writers.

CopyOnWriteArrayList

Copies on every write. Great for rarely changed lists.

BlockingQueue

Producers wait when full, consumers wait when empty.

Overview only: threads come later in the course

Demo: Unmodifiable Collections and Thread Safety

What we build

- **View and copy:** the cart view follows its owner, the receipt is a snapshot.
- **Read-only:** changing a view or List.of throws an exception.
- **Two counters:** count laptops sold together in a ConcurrentHashMap.

How the concept helps

- **Unmodifiable** stops callers from damaging our data.
- **Snapshot** protects a receipt from later cart changes.
- **Concurrent collections** stay correct when threads share them.

Demo: Building Our Own Data Structure

What we build

- **LinkedStack:** a generic stack built from linked nodes.
- **Iterable:** the stack works in for-each, newest first.
- **Cart undo:** push each change, pop to undo it.

How the concept helps

- **Custom structure** shows how the standard collections work inside.
- **Generics** let one stack hold strings, numbers or products.
- **Iterable** makes our class feel like a built-in collection.

Key Takeaways

1. **Hierarchy**
   List, Set, Queue, Deque; Map is separate.

2. **Choose by need**
   ArrayList, HashMap, HashSet by default; Linked or Tree when order matters.

3. **Ordering**
   Comparable once, Comparator many, chain with thenComparing.

4. **Safe loops**
   Iterator.remove() or removeIf, never edit inside for-each.

5. **Protect data**
   Return unmodifiable views; use List.of for constants.



Happy Learning :)